

Simplified BitTorrent-like P2P File Sharing System

Technical Project Report

Project author	AydinEZP
Implementation	Python standard library
Communication model	HTTP tracker discovery and direct TCP peer transfer
Validation scope	Static review and isolated local test execution

This report documents the supplied implementation and supported results. Details absent from the project materials are not inferred.

Abstract

This project implements a compact peer-to-peer file-sharing system inspired by the separation of responsibilities in BitTorrent. A multithreaded HTTP tracker maintains swarm membership and returns reachable peers, while file data is transferred directly between peers over TCP. Human-readable JSON metainfo describes single-file or multi-file payloads, piece boundaries, and SHA-1 hashes. Tracker responses use Bencode, whereas the custom peer protocol frames JSON messages with a four-byte network-order length header and carries piece bytes in Base64 form.

The implementation includes metainfo generation and validation, synchronized tracker state, peer discovery, PING/PONG reachability checks, bitfield exchange, piece requests, cross-file piece mapping, hash verification, independent torrent workers, and thread-safe event logging. The supplied report records successful byte-for-byte reconstruction of a 4,099-byte single-file payload and a 3,164-byte multi-file bundle. The project also includes component and integration tests that exercise the tracker, peer protocol, piece transfer, concurrent workers, and complete local workflow. The system is educational and deliberately omits several production BitTorrent policies and Internet-deployment concerns.

Contents

1	Project Overview and Objectives	1
1.1	Objectives	1
1.2	Implementation inventory	1
2	System Architecture	1
2.1	Torrent lifecycle	2
3	Metainfo and Tracker Design	2
3.1	Human-readable metainfo	2
3.2	Deterministic information hash	2
3.3	Tracker state and HTTP interface	3
4	Peer Protocol and Direct Transfer	3
4.1	Message framing	3
4.2	Reachability and availability	3
4.3	Piece exchange	3
5	Piece Management, Concurrency, and Logging	4
5.1	Virtual file layout	4
5.2	Concurrent services	4
5.3	Event logging	4
6	Validation Methodology	5
7	Results and Observations	5
8	Limitations and Reproducibility	5
8.1	Protocol limitations	5
8.2	Reproduction procedure	7
8.3	Publication boundaries	7
9	Conclusion	7
A	Execution Guide	9
A.1	Automated validation	9
A.2	Automated integrated demo	9
A.3	Manual Windows workflow	9

List of Figures

1	Control-plane discovery through the tracker and direct peer-to-peer data transfer.	2
2	Loopback capture of HTTP announce connections between peers and the tracker. File payloads are not present on this tracker connection.	6
3	Loopback capture of direct TCP connections on the configured peer ports.	6
4	Packet-capture evidence for the custom peer reachability exchange.	7

List of Tables

1	Principal implementation components in the supplied project.	1
2	Test groups executed by the supplied runner.	5

1 Project Overview and Objectives

The project demonstrates a central architectural property of peer-to-peer file sharing: discovery and control information can be coordinated by a tracker without routing the transferred file through that tracker. The implementation is a simplified, locally runnable system based on requirements described in the course project handout (Course staff, n.d.) and protocol concepts referenced from BEP 3 (BitTorrent.org, n.d.). It is not intended to be a compatible replacement for a standard BitTorrent client.

1.1 Objectives

The implemented workflow addresses five concrete objectives:

1. describe single-file and multi-file payloads as pieces with expected hashes;
2. register peers with a tracker and return swarm membership and statistics;
3. exchange framed control and piece messages directly between peers;
4. verify each received piece before making it available locally; and
5. support concurrent torrent workers with auditable event logs.

1.2 Implementation inventory

Table 1: Principal implementation components in the supplied project.

Component	Responsibility
Metainfo parser	Validates JSON metainfo, normalizes single-file and multi-file layouts, and calculates the torrent information hash.
Tracker server and state	Processes HTTP announce requests, records peers by torrent, removes expired peers, and returns Bencoded responses.
Tracker client	Encodes binary identifiers into announce URLs and parses tracker responses.
Peer server and client	Exchanges reachability, availability, interest, and piece messages over framed TCP connections.
Piece manager	Maps a torrent to local files, scans available pieces, verifies SHA-1 hashes, and reads or writes piece ranges.
Torrent workers	Runs independent transfer lifecycles and periodic tracker announcements for multiple torrents.
Event logger	Serializes timestamped tracker and peer events using per-path locks and atomic file replacement.

The code depends only on Python standard-library modules such as `http.server`, `socket`, `threading`, and `hashlib` (Python documentation, n.d.). Type-union syntax used by the source requires Python 3.10 or newer.

2 System Architecture

Each peer reads the metainfo locally. The tracker receives HTTP announcements and returns peer endpoints; it never serves file pieces. Direct TCP connections carry peer-protocol messages and piece data. This separation is summarized in figure 1.

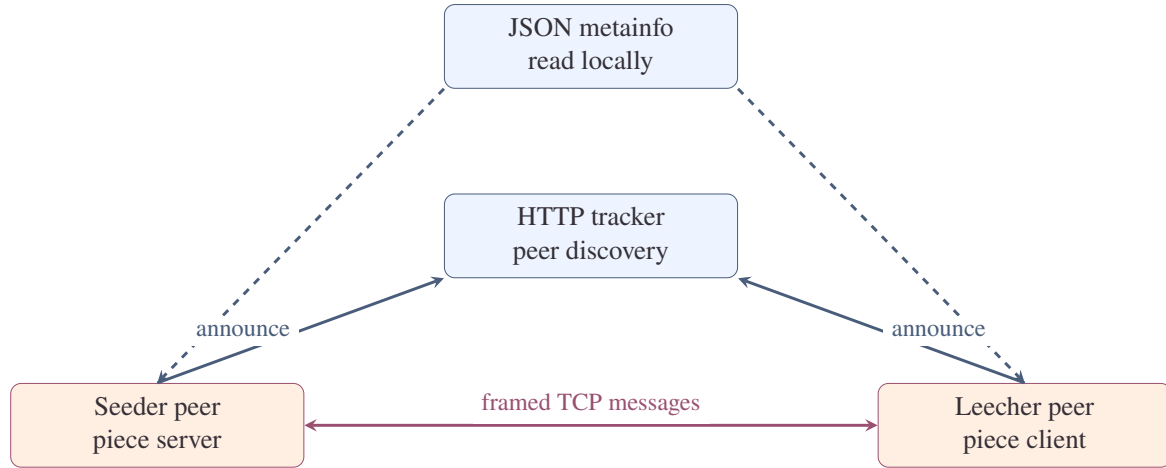


Figure 1: Control-plane discovery through the tracker and direct peer-to-peer data transfer.

2.1 Torrent lifecycle

For each metainfo file, a worker performs the following sequence:

1. load and validate the metainfo;
2. scan local storage to establish the initial bitfield;
3. start a peer server on a dedicated port;
4. announce the `started` event to the tracker;
5. discover peers and probe them with PING/PONG;
6. request availability, express interest, and download missing pieces;
7. verify every piece before storing it;
8. send periodic progress announcements; and
9. send `completed` and `stopped` events when applicable.

Multiple metainfo files are represented as separate jobs in a thread pool. Their storage managers, network listeners, and transfer statistics remain distinct while they share the same application process.

3 Metainfo and Tracker Design

3.1 Human-readable metainfo

The project stores metainfo in JSON rather than the binary `.torrent` format. The `announce` field contains either one tracker URL or a list whose first entry is selected. The `info` dictionary contains the payload name, piece length, piece hashes, and either one total length or a list of files.

For total payload length L_{total} and piece length L_{piece} , the expected number of pieces is

$$N_{\text{pieces}} = \left\lceil \frac{L_{\text{total}}}{L_{\text{piece}}} \right\rceil. \quad (1)$$

In multi-file mode, the listed files form one virtual byte stream. A piece may therefore begin near the end of one file and continue in the next. The parser validates required fields, positive lengths, hash syntax, piece count, and path components. Absolute paths, parent-directory traversal, and unsafe separators are rejected.

3.2 Deterministic information hash

The `info` dictionary is Bencoded with sorted dictionary keys and hashed with SHA-1:

$$\text{info_hash} = \text{SHA1}(\text{Bencode}(\text{info})) . \quad (2)$$

This keeps torrent identity independent of the outer JSON layout while preserving deterministic encoding for the implemented field types.

3.3 Tracker state and HTTP interface

The tracker uses Python's `ThreadingHTTPServer`. Announce requests provide the information hash, peer identifier, listening port, uploaded and downloaded counters, remaining bytes, and an optional event. Binary identifiers are decoded from percent-encoded URL bytes.

Tracker state is grouped first by `info_hash` and then by `peer_id`. An `RLock` protects concurrent updates, while a cleanup thread removes peers that have not announced within the configured timeout. Successful responses contain the announce interval, complete and incomplete counts, and the peer list in Bencoded form.

The tracker is a discovery and state-coordination service. The implementation does not read, cache, or relay the shared payload.

4 Peer Protocol and Direct Transfer

4.1 Message framing

TCP presents a byte stream rather than message boundaries. The implementation prefixes each JSON body with a four-byte unsigned length in network byte order:

$$\text{frame} = \text{uint32_be}(|\text{payload}|) \parallel \text{JSON payload} . \quad (3)$$

The receive routine repeatedly reads until the declared number of bytes has arrived, so a message split across multiple TCP packets is reconstructed before JSON decoding. The default maximum frame size is eight MiB. The peer vocabulary includes PING, PONG, BITFIELD, INTERESTED, UNCHOKE, REQUEST, PIECE, and ERROR.

4.2 Reachability and availability

PING messages contain a random nonce and sender identifier. A valid PONG returns the same nonce, which lets the initiating peer associate the response with its probe. Once a peer is reachable, the client requests its bitfield, expresses interest when useful pieces are available, and waits for an unchoke response before requesting a piece.

4.3 Piece exchange

Piece bytes are Base64-encoded in the JSON body. This is easy to inspect but adds encoding overhead compared with a binary peer protocol. Algorithm 1 summarizes the implemented validation cycle.

Algorithm 1 Validated piece request cycle

Require: peer endpoint, local piece manager, missing piece index i

```

1: connect to the peer and request its bitfield
2: if piece  $i$  is unavailable then
3:     close the connection and select another peer
4: end if
5: send INTERESTED and require UNCHOKE
6: send REQUEST for piece  $i$ 
7: decode the returned Base64 piece bytes
8: verify the declared index, length, and expected SHA-1 hash
9: if verification succeeds then
10:    store the piece and update session counters
11: else
12:    reject the bytes and log HASH_INVALID
13: end if

```

5 Piece Management, Concurrency, and Logging

5.1 Virtual file layout

The piece manager converts the metainfo layout into a sequence of file segments with absolute offsets in the virtual torrent stream. Reads and writes are intersected with these segments, allowing one piece operation to span several local files. On startup, the manager scans existing storage and marks a piece available only when both its length and SHA-1 value match the metainfo.

For received bytes p_i and expected hash h_i , acceptance requires

$$\text{SHA1}(p_i) = h_i. \quad (4)$$

A mismatch raises a dedicated error and leaves the piece unavailable. This check supports integrity within the project workflow; it does not authenticate the source peer.

5.2 Concurrent services

The tracker handles HTTP requests on separate threads and synchronizes shared swarm state. Each peer server accepts TCP connections and dispatches client handling concurrently. At application level, the torrent thread pool creates one worker per metainfo job, assigns a listening port, and tracks progress independently. A stop event coordinates graceful shutdown.

The design separates two measurements:

- the piece manager represents durable availability inferred from local files; and
- transfer statistics represent bytes uploaded and downloaded during the current worker session.

This distinction prevents pre-existing valid pieces from being reported as bytes downloaded in the current run.

5.3 Event logging

The event logger records a file header followed by timestamped event | time | description rows. Inputs are normalized to one line, access is guarded by a per-path re-entrant lock, and updates are written to a temporary file before atomic replacement. Tracker and peer components use event names such as ANNOUNCE_RECEIVED, PING_SENT, PIECE_RECEIVED, HASH_VALID, and PIECE_STORED.

Table 2: Test groups executed by the supplied runner.

Test group	Primary coverage
Metainfo	Single-file and multi-file creation, field validation, path safety, piece hashes, and cross-file boundaries.
Logger	Header validation, concurrent writers, field sanitization, and atomic updates.
Tracker state	Registration, completion, stopping, peer counts, timeout cleanup, and concurrent updates.
Tracker HTTP	Announce parsing, binary URL values, response encoding, errors, and concurrent requests.
Tracker client	URL generation, Bencode parsing, peer filtering, and progress events.
Peer PING	Framing, partial reads, message-size limits, PING/PONG, and reachability results.
Piece transfer	Bitfields, piece requests, valid storage, corrupt-piece rejection, and multi-file reconstruction.
Torrent thread pool	Independent workers, concurrent downloads, progress accounting, and shutdown.
Final integration	Demo preparation, two-torrent transfer, byte comparison, repository audit, and submission-archive checks.

6 Validation Methodology

The repository provides a standard-library-only test runner, `run_all_tests.py`. Tests create deterministic local payloads, bind tracker and peer services to loopback addresses, and write disposable outputs under ignored directories. No external tracker, dataset, or API is required.

The validation evidence is functional rather than performance-oriented. The materials do not specify throughput benchmarks, wide-area latency experiments, stress-test scale, or comparative baselines. Accordingly, no performance or scalability claim is made.

7 Results and Observations

The supplied report records PASS outcomes for metainfo, tracker, peer, piece-transfer, multi-torrent, and integration checks. The integrated scenario used two torrents: a 4,099-byte single-file payload and a 3,164-byte multi-file bundle. The total transferred payload was therefore

$$4,099 + 3,164 = 7,263 \text{ bytes.} \quad (5)$$

The integration assertions require both leecher workers to finish, both seeder workers to remain complete, session upload and download totals to equal 7,263 bytes, and three output files to match their sources. These checks validate the demonstrated workflow but should not be interpreted as a throughput benchmark.

Figures 2 and 3 visually separates tracker traffic on port 8000 from direct traffic on peer ports. Figure 4 shows the four-byte frame prefix followed by JSON application data and matching nonce values across PING/PONG. Packet segmentation itself is not used as a correctness criterion; application-level length, piece index, and hash checks provide the relevant validation.

8 Limitations and Reproducibility

8.1 Protocol limitations

The implementation adopts selected BitTorrent concepts but defines its own metainfo and peer wire formats. JSON metainfo is not a standard binary `.torrent` file, and peer messages are length-prefixed

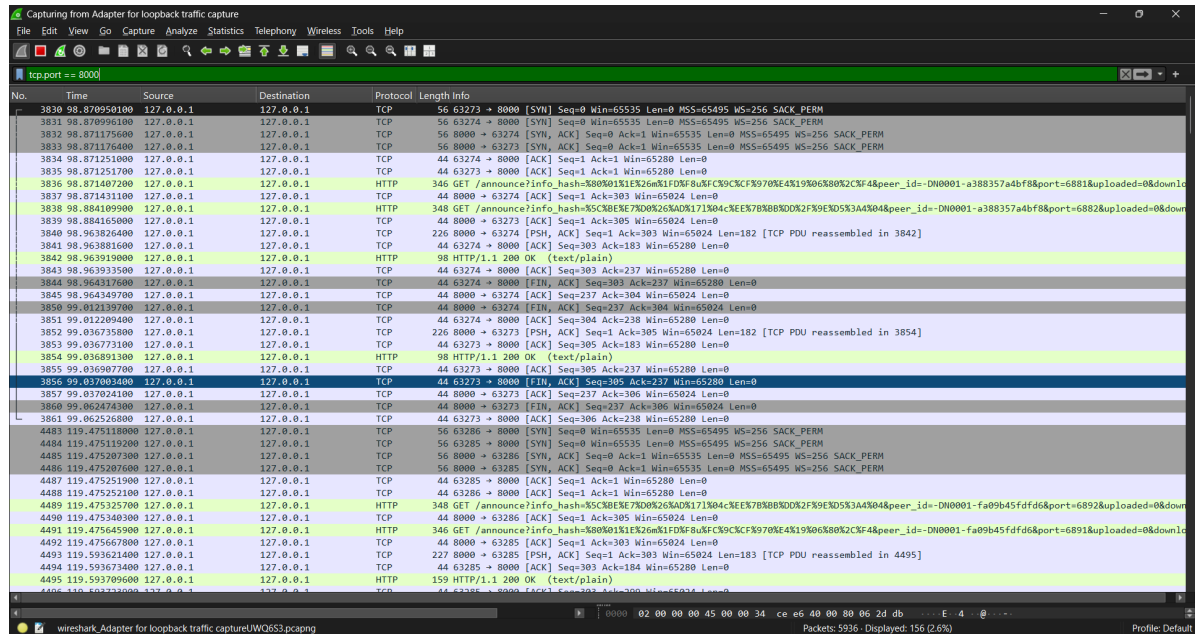


Figure 2: Loopback capture of HTTP announce connections between peers and the tracker. File payloads are not present on this tracker connection.

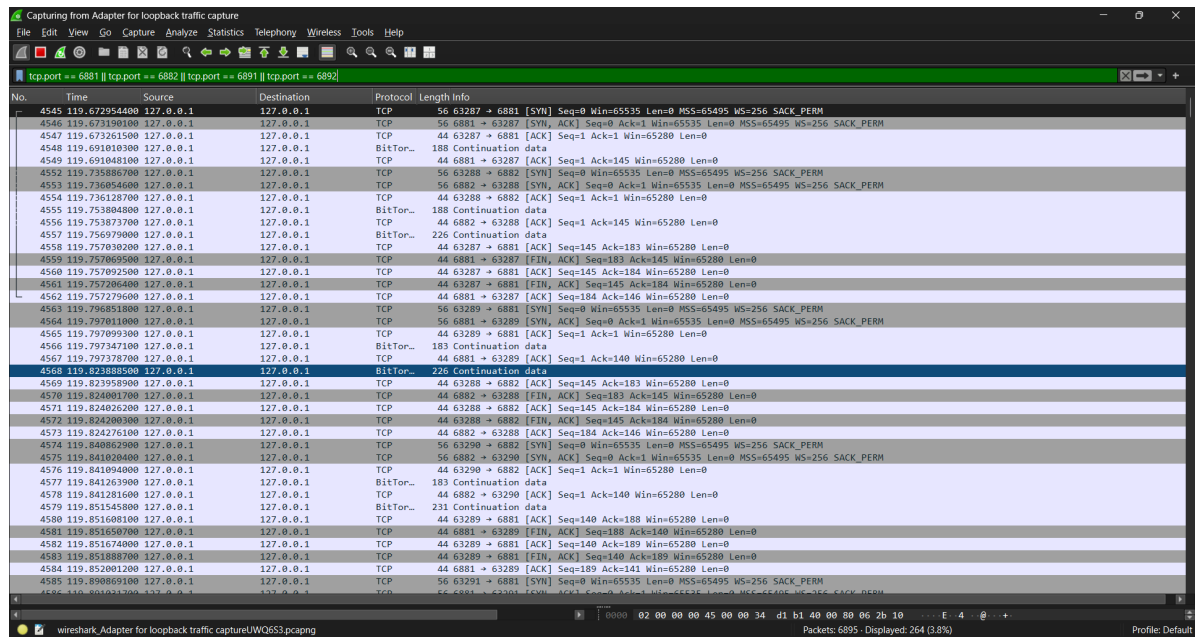
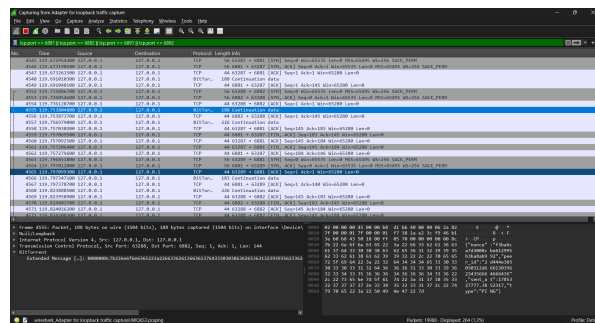
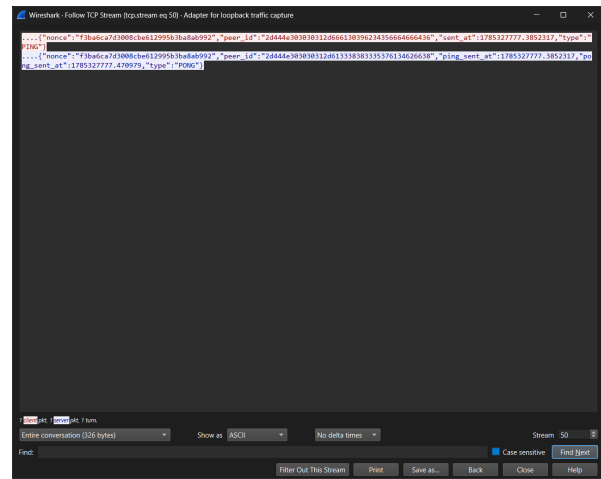


Figure 3: Loopback capture of direct TCP connections on the configured peer ports.



(a) Framed PING message in one peer connection.



(b) PING and PONG payloads in a followed TCP stream.

Figure 4: Packet-capture evidence for the custom peer reachability exchange.

JSON with Base64 piece data. The project does not implement rarest-first selection, end-game mode, tit-for-tat, NAT traversal, transport encryption, authentication, or a distributed hash table. Tracker and peer demonstrations use loopback networking.

SHA-1 verifies that a piece matches the metainfo, but the implementation does not establish who produced the metainfo or authenticate a remote peer. The supplied materials do not document an adversarial threat model, Internet deployment, sustained-load testing, or resource-isolation policy. These omissions limit any security or scalability interpretation.

8.2 Reproduction procedure

The implementation has no third-party Python dependency. A reproduction run requires Python 3.10 or newer and permission to bind local loopback ports. From the repository root, the complete test sequence is launched with:

```
python run_all_tests.py
```

The tests create their own small payloads and output trees. Generated data, logs, caches, and submission archives are ignored by version control. The supplied materials do not specify the original Python patch history or a complete operating-system configuration, so exact-environment equivalence cannot be claimed.

8.3 Publication boundaries

The release excludes generated outputs, the redundant nested submission archive, bytecode caches, bundled fonts with unclear redistribution terms, the old PDF export, and six screenshots that display an absolute local path. Four project-generated loopback packet-capture screenshots are retained because they support the network-flow discussion without exposing the student identifier, email, credential, or private filesystem location.

9 Conclusion

The project provides a coherent educational implementation of tracker-assisted peer-to-peer file sharing. It separates peer discovery from payload transport, supports single-file and multi-file metainfo, frames peer messages over TCP, verifies each piece before storage, and coordinates several torrent workers concurrently.

The test suite covers individual parsing and state components as well as complete loopback transfer and byte-for-byte reconstruction.

The implementation's strongest contribution is the integration of these concerns in a standard-library-only codebase with explicit event logging and reproducible local tests. Its scope is deliberately narrower than BitTorrent: custom JSON formats, loopback validation, and omitted production policies make it suitable as a networking course project rather than a general-purpose client.

References

BitTorrent.org. Bittorrent enhancement proposal 3: The bittorrent protocol specification. bittorrent.org, n.d. Referenced by the supplied project report.

Course staff. Final project specification for data communications networks, n.d. Course handout referenced by the supplied project report; publication metadata was not specified.

Python documentation. Python standard library documentation for `http.server`, `socket`, `threading`, and `hashlib`, n.d. Version and access date were not specified in the supplied project materials.

A Execution Guide

A.1 Automated validation

Run all nine test groups from the repository root:

```
python run_all_tests.py
```

The runner creates `test_data/` and `test_output/`; both are generated and ignored by Git.

A.2 Automated integrated demo

The integrated demo starts a loopback tracker, prepares deterministic single-file and multi-file payloads, launches seeder and leecher workers, and verifies the reconstructed files:

```
python -m tools.end_to_end_demo \  
  --workspace demo_workspace_integrated \  
  --reset
```

A.3 Manual Windows workflow

The `demo/` directory contains five command files. Run them in numeric order. The tracker, seeder, and leecher commands are intentionally separated so their logs and progress can be observed in different terminals.

```
01_prepare_demo.cmd  
02_run_tracker.cmd  
03_run_seed_peer.cmd  
04_run_leecher_peer.cmd  
05_verify_demo.cmd
```

All default endpoints use `127.0.0.1`. The tracker defaults to port 8000; the manual seeder and leecher commands begin their peer-port ranges at 6881 and 6891, respectively.